

The Cartographer Role: Situational Awareness Through Maps

Document: issues-fs__cartographer-role

Version: v1.0

Date: 2026-02-07

Status: Draft

Depends On: issues-fs__thinking-in-graphs v1.0, issues-fs__lexicon-architecture v2.0, issues-fs__role-based-agent-coordination v1.0, issues-fs__librarian-role v1.0

What This Document Is

This document defines the Cartographer role for the Issues-FS ecosystem. Where the Librarian creates meaning through connectivity and the Architect defines boundaries through decisions, the Cartographer creates understanding through position. Its primary instrument is the Wardley Map — a visual representation of a value chain plotted against evolution — and its primary output is situational awareness: the ability to see where things are, where they are going, and what that implies for strategy.

The central claim: **maps are the natural evolution of graphs**. A graph says "these things are connected." A map says "these things are connected, *and here is where they sit relative to each other and relative to the landscape*." In a system built on the principle that meaning comes from connectivity, the Cartographer adds a second dimension of meaning: position. Connectivity tells you *what relates to what*. Position tells you *what to do about it*.

This is not a visualisation role. It is a strategic intelligence role that happens to produce visual artifacts.

Part 1: Why Wardley Maps

The Problem Maps Solve

The Issues-FS ecosystem has graphs that describe what exists and how it connects. The Lexicon provides anchor nodes. The Librarian maintains connectivity. The analysis tools compute confidence and compatibility. But none of these answer the questions that drive strategic decisions:

- What should we build next?
- What should we stop building and buy or reuse?
- Where are we investing effort in things that should be commodities?
- Where are we under-investing in things that are genuinely novel?
- What will the landscape look like in six months?

These are questions of *position* and *movement*. They require not just knowing what exists, but knowing where each component sits on an evolutionary axis — and what that position implies.

Wardley Maps answer these questions by plotting components on two axes: **visibility** (how close a component is to a user need) and **evolution** (how mature the component is, from Genesis through Custom-Built, Product, to Commodity). The resulting map is a landscape that makes strategic options visible.

Wardley Maps as Graph Extensions

Here is the key architectural insight: **a Wardley Map is not a separate artifact from the graph. It is a graph with positional metadata.**

Every component on a Wardley Map is a node. Every dependency line is an edge. What makes it a map rather than just a graph is two additional pieces of information per node:

1. **Evolution position** — where on the Genesis→Commodity axis this node sits
2. **Visibility position** — how many hops from a user need this node is (derived from the dependency chain)

In graph terms, converting a subgraph into a Wardley Map requires adding two edges per node:



That's it. A Wardley Map is literally a graph view that reads these two additional edges and renders nodes spatially. The graph is primary; the map is a projection. This means:

- Maps inherit all the properties of graphs: fractal scoping, anchor nodes, connectivity analysis, honest uncertainty
- Map data lives in the same graph infrastructure as everything else
- Map queries are graph queries with positional filters
- Maps compose the same way graphs compose: maps of maps of maps

Beyond the Standard Evolution Axis

The standard Wardley Map uses a single evolution axis: Genesis → Custom-Built → Product → Commodity. This is powerful but not the only useful axis. The Cartographer should explore alternative evolution axes that reveal different strategic dimensions:

Axis	Left (less evolved)	Right (more evolved)	Reveals
Standard evolution	Genesis	Commodity	Build vs buy decisions
Openness	Closed/proprietary	Open/standard	Lock-in risk, interoperability
Automation	Manual/human	Fully automated	Agent readiness, scaling potential
Documentation	Undocumented	Fully documented	Onboarding cost, bus factor
Test coverage	Untested	Fully tested	Reliability, change confidence
Graph connectivity	Isolated node	Richly connected	Meaning confidence (directly from thinking-in-graphs)

The same set of components can be plotted against different axes to reveal different strategic pictures. A component that is "Product" on the standard axis might be "Closed" on the openness axis — suggesting a strategic tension worth surfacing.

This is where Issues-FS can serve as a demonstration of how Wardley Mapping extends beyond its typical application: by treating the evolution axis as a configurable dimension rather than a fixed one, and by storing axis metadata in the graph alongside everything else.

Part 2: The Cartographer's Responsibilities

Primary Responsibility: Create and Maintain Maps

The Cartographer's core job is to produce and keep alive a set of Wardley Maps that give the team situational awareness. "Keep alive" is critical — a map that was accurate three months ago and hasn't been updated is worse than no map, because it creates false confidence.

The Cartographer runs on a regular cadence — daily or per-sprint — reviewing maps against the current state of the ecosystem and updating positions, adding new components, removing retired ones, and flagging movements.

Initial map set:

1. **The Landscape Map** — "What is Issues-FS?" This is the root map. It starts with user needs (manage projects, coordinate agents, track issues) and traces the dependency chain down through visible components (UI, CLI, API), through the service layer, through core library and graph engine, down to infrastructure (Git, Python, PyPI). Every component is positioned on the evolution axis. This map answers: "what are we building, and where does each piece sit on the maturity curve?"
2. **Role Maturity Maps** — One map per role, showing the components of that role (definition, workflows, tooling, prompts, issue types) plotted against evolution. A newly defined role like Cartographer has thin, left-leaning map (Genesis: barely defined). A mature role like Dev has a wider, right-leaning map (Custom-Built to Product: well-defined workflows, established patterns). These maps make role investment decisions visible.
3. **Competitive Comparison Maps** — Issues-FS vs GitHub Issues, vs Jira, vs Linear. Same user needs, different component positions. These maps reveal where Issues-FS is differentiated (graph-native, file-based, agent-coordinated) vs where it is behind (UI maturity, integrations, market presence).
4. **Per-Project Maps** — For each active project or epic, a map showing the components involved and their evolution. These are working maps — updated as implementation progresses.

Second Responsibility: Doctrine Assessment

Wardley Mapping defines a set of **doctrine** — universal principles that apply regardless of strategy. These include:

- Use a common language
- Challenge assumptions

- Focus on user needs
- Use appropriate methods
- Think small (as in teams)
- Be transparent
- Move fast
- Be pragmatic
- Remove bias and duplication
- Use standards where appropriate
- Manage inertia
- Optimise flow
- Effectiveness over efficiency
- Think aptitude and attitude

The Cartographer's doctrine responsibility is to periodically evaluate the Issues-FS ecosystem against these principles and produce a score or health assessment. This is analogous to the Librarian's ecosystem health scan, but focused on strategic health rather than knowledge health.

The doctrine assessment is fractal: it can be applied to the ecosystem as a whole, to a specific project, to a role, or to a single sprint. At each level, the Cartographer asks: "Are we following doctrine here? Where are we strong? Where are we weak? What's the risk?"

Third Responsibility: Gameplay Identification

Beyond maps and doctrine, Wardley Mapping defines a rich set of **gameplays** — strategic moves that exploit landscape positions. These include:

Gameplay	Description	Potential Issues-FS Application
ILC (Innovate-Leverage-Commoditise)	Create something novel, leverage it for advantage, then commoditise it to undermine competitors	The coordination protocol is Genesis — could it become a commodity that makes other issue trackers depend on it?
Open approaches	Open-source a component to accelerate evolution and build ecosystem	Issues-FS is already open — but are all the right components open?
Ecosystem play	Build a platform that others build on	The Lexicon as a shared vocabulary that other tools could adopt
Tower and moat	Invest in a key differentiator and defend it	Graph-native issue tracking as the tower; the thinking-in-graphs philosophy as the moat
Red Queen	Run faster just to stay in the same place — recognise when you're in a Red Queen situation and break out	Are any components stuck in competitive catch-up?
Sensing engines	Build mechanisms to detect change early	The Cartographer itself is a sensing engine

The Cartographer's gameplay responsibility is to identify which gameplays are available given the current landscape, which are being played (intentionally or accidentally), and which should be considered. This is strategic advisory work that feeds into the Architect's decisions and the Conductor's planning.

Fourth Responsibility: Change Impact Mapping

As the ecosystem matures, every significant change — a PR, a release, a new dependency, an architectural decision — shifts the landscape. The Cartographer's change impact responsibility is to assess how changes affect the maps:

- Which components moved on the evolution axis?
- Did a new dependency appear? Where does it sit?
- Did a component get commoditised (replaced by a library or standard)?
- Did a custom-built component become more Genesis (we discovered it's harder than we thought)?

Eventually, this becomes a regular artifact: every significant change comes with a map diff — a before/after showing what moved and what the movement implies. This is the strategic equivalent of a code diff: it shows not what changed in the code, but what changed in the landscape.

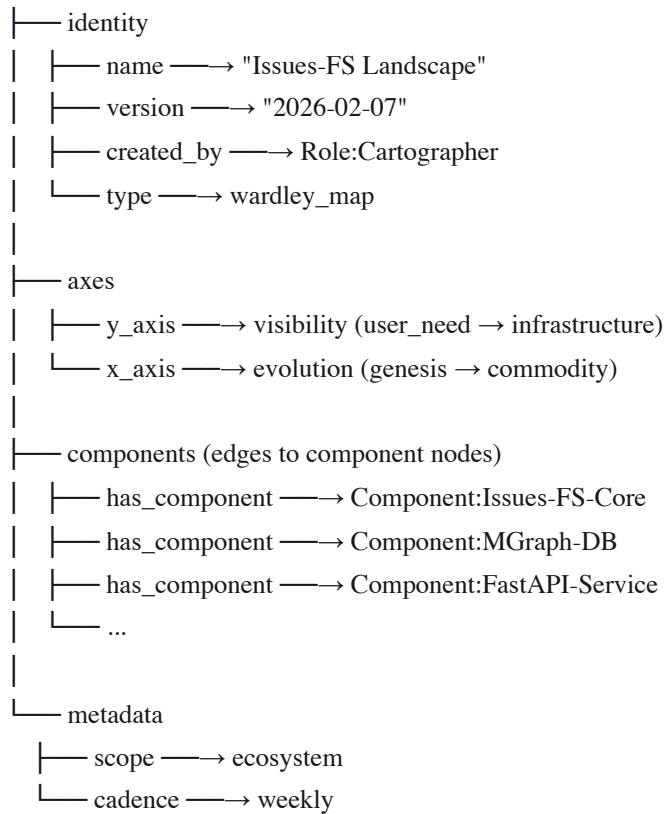
Part 3: Maps as Graphs — The Data Model

Storing Maps in the Graph

Since maps are graphs with positional metadata, map data lives in the Issues-FS graph. The Cartographer does not maintain a separate mapping tool. Map data is stored as nodes and edges that the existing graph infrastructure can query, version, and analyse.

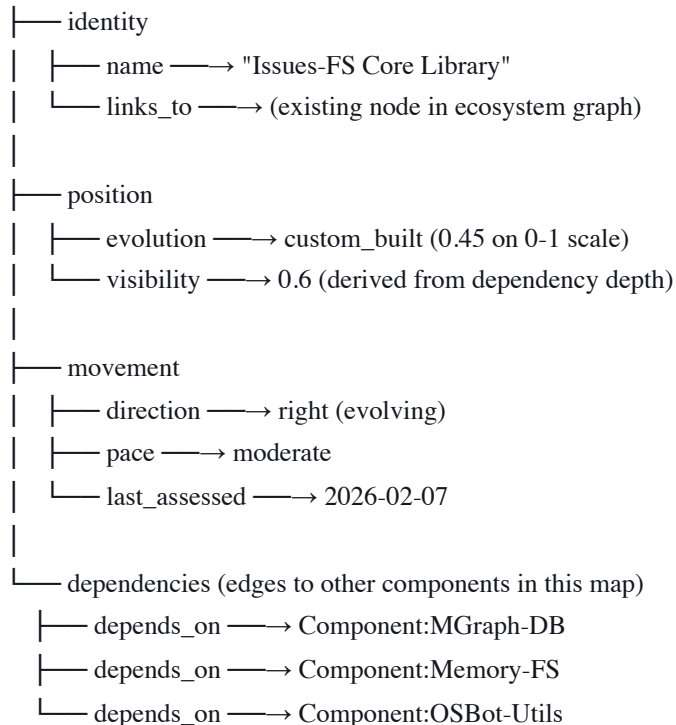
Map node:

Map:Issues-FS-Landscape



Component node (map-specific):

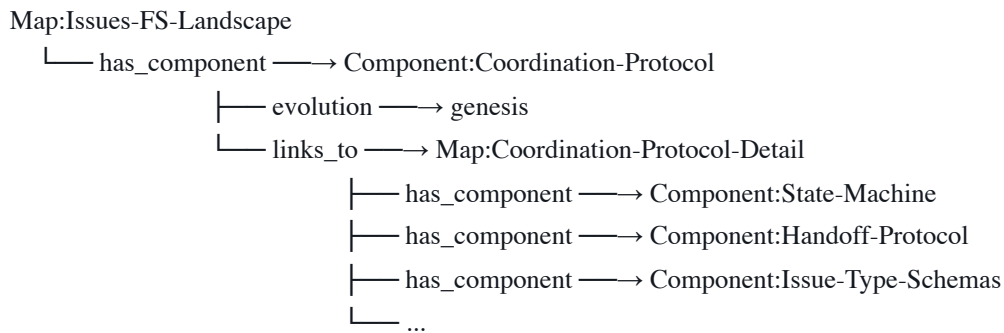
Component:Issues-FS-Core



The critical design choice: **component nodes in a map link back to existing nodes in the ecosystem graph.** The map does not duplicate the ecosystem — it adds a positional projection of it. The `links_to` edge connects the map component to the actual node (a repo, a role, a concept), so map queries can traverse into the ecosystem graph and ecosystem queries can traverse into the maps.

Maps of Maps

The fractal principle applies directly. Every component on a map can itself be a map:



The top-level map shows the Coordination Protocol as a single dot positioned at Genesis. Drilling into that dot reveals its own map with its own components at their own evolution positions. The position of the parent component in the parent map should be consistent with the aggregate position of its children: if the sub-map shows mostly Genesis components, the parent dot should sit at Genesis.

This is maps of maps of maps — and since maps are graphs, it is graphs of graphs of graphs, which is exactly the recursive structure described in the thinking-in-graphs document.

Map Ontology and Taxonomy

Maps within the same domain should share a common ontology — the same node types, link types, and axes — while differing in which nodes exist and where they are positioned. This means:

Shared across all Wardley Maps:

- Node type: `map_component`
- Edge types: `depends_on`, `evolution_position`, `visibility_position`, `map_membership`
- Evolution stages: `genesis`, `custom_built`, `product`, `commodity` (with numeric positions for finer granularity)
- Movement annotations: `direction` (left/right/stable), `pace` (fast/moderate/slow), `inertia` (yes/no)

Varies per map:

- Which components are included
- The positions of those components
- The dependency edges between them
- The x-axis definition (standard evolution vs alternative axes)

This is a direct application of the thinking-in-graphs principle that meaning comes from edges, not from node properties. Two maps that share the same ontology are structurally comparable: the Cartographer can compute diffs, overlaps, and divergences between them using the same graph analysis tools the Librarian uses for documents.

Part 4: Integration with Existing Roles

Cartographer ↔ Librarian

These two roles are complementary and will collaborate closely. The Librarian organises knowledge; the Cartographer positions it strategically. Specific integration points:

- **Shared storage concerns** — Both roles produce graph-native artifacts. The Cartographer should work with the Librarian on data storage strategies, conventions for map versioning, and cataloguing of map artifacts.
- **Map data as catalogued artifacts** — Every map the Cartographer produces is a knowledge artifact that the Librarian should accession, catalogue, and cross-reference.
- **Librarian health scans informed by maps** — When the Cartographer identifies a component at Genesis with poor documentation, the Librarian can prioritise its cataloguing.
- **Cartographer maps informed by Librarian data** — The Librarian's connectivity analysis (how well-connected a node is) can feed directly into alternative evolution axes (graph connectivity as an evolution dimension).

Cartographer ↔ Architect

The Architect makes strategic technical decisions. The Cartographer provides the landscape context for those decisions. Specific integration points:

- **Decision context** — Every ADR created by the Architect should reference the relevant map. "We chose WebSocket over polling" makes more sense when you can see that WebSocket sits at Product on the evolution axis while the real-time sync capability it enables sits at Genesis.
- **Dependency evaluation** — When the Architect evaluates a new dependency, the Cartographer can show where it sits on the evolution axis. A Genesis dependency in a commodity position is a strategic risk.
- **Gameplay recommendations** — The Cartographer identifies available gameplays; the Architect decides which to pursue via Decision issues.

Cartographer ↔ Conductor

The Conductor orchestrates workflow; the Cartographer provides strategic context for prioritisation. Specific integration points:

- **Sprint planning** — Maps inform what to invest in. Components at Genesis need exploration (spikes, prototypes). Components at Custom-Built need engineering. Components at Product need polish. Components at Commodity should be replaced with off-the-shelf solutions.
- **Risk visibility** — Maps reveal dependencies on immature components, concentration of Genesis-stage work, and strategic gaps that should influence priority.
- **Daily/weekly situational awareness** — The Cartographer's regular map updates feed the Conductor's understanding of where the project stands and where it is heading.

Cartographer ↔ Dev, QA, DevOps

For execution roles, the Cartographer provides context that informs approach:

- **Dev** — A component at Genesis needs different engineering practices (exploratory, prototype-friendly) than a component at Product (stable interfaces, backward compatibility). Map position informs coding approach.
 - **QA** — Test strategy should match evolution stage. Genesis components need exploratory testing. Product components need regression suites. Map position informs test investment.
 - **DevOps** — Deployment complexity should match evolution stage. Genesis components can have rough deployment. Commodity components should have fully automated pipelines. Map position informs infrastructure investment.
-

Part 5: Workflows

Workflow 1: Initial Landscape Mapping

When the Cartographer role is first activated:

1. Identify user needs

- |— What problems does Issues-FS solve?
- |— Who are the users? (developers, project managers, agents)
- |— What are their top-level needs? (manage projects, coordinate agents, track issues, query relationships)

2. Enumerate components

- |— Walk the ecosystem graph (repos, services, libraries, tools)
- |— Walk the role graph (roles, workflows, protocols)
- |— Walk the concept graph (Lexicon anchors, patterns)

3. Position each component

- |— Evolution: genesis / custom-built / product / commodity
- |— Visibility: distance from user need in the dependency chain
- |— Movement: is this evolving? In which direction? How fast?

4. Draw dependency edges

- |— Which components depend on which?
- |— Validate against the actual dependency graph in the ecosystem

5. Identify strategic observations

- |— Where are we investing in genesis (high risk, high potential)?
- |— Where are we custom-building what should be commodity (waste)?
- |— Where are dependencies on immature components (fragility)?
- |— Where are the differentiators (what makes Issues-FS unique)?

6. Publish the map

- |— Store map data in the graph (nodes + positional edges)
- |— Generate visual rendering
- |— Create a Knowledge_Request for the Librarian to catalogue
- |— Present findings to the Conductor and Architect

Workflow 2: Regular Map Maintenance

On a regular cadence (daily or per-sprint):

1. Review each active map

- |— Have any components moved on the evolution axis?
- |— Have new components appeared (new repos, new dependencies)?
- |— Have any components been retired or replaced?
- |— Have any dependency relationships changed?

2. Update positions

- |— Adjust evolution positions based on current state
- |— Annotate movements with rationale
- |— Flag significant shifts for Conductor/Architect attention

3. Generate diffs

- |— Compare current map to previous version
- |— Highlight what moved, what appeared, what disappeared
- |— Summarise strategic implications of the changes

4. Publish updates

- |— Version the map (new snapshot in the graph)
- |— Update visual rendering
- |— Notify Conductor if strategic implications warrant attention

Workflow 3: Doctrine Assessment

Periodically (per-milestone or quarterly):

1. Select scope (ecosystem, project, role, sprint)
2. Evaluate against each doctrine principle
 - |— Common language: Are we using consistent terminology?
 - | (Cross-reference with Librarian's authority control data)
 - |— Challenge assumptions: Are we questioning positions?
 - |— Focus on user needs: Do our maps start from user needs?
 - |— Use appropriate methods: Are we matching approach to evolution?
 - |— Think small: Are teams/roles focused and bounded?
 - |— Be transparent: Is the landscape visible to all roles?
 - |— Move fast: Are Genesis components being explored rapidly?
 - |— Be pragmatic: Are we building vs buying appropriately?
 - |— Remove duplication: Are there redundant components?
 - |— Use standards: Are commodity components using standards?
 - |— Manage inertia: Are we stuck on custom-built where we should commoditise?
 - |— Optimise flow: Are handoffs between roles efficient?
 - |— Effectiveness over efficiency: Are we doing the right things?
 - |— Think aptitude and attitude: Do role assignments match component evolution?
3. Score and report
 - |— Produce a doctrine health score (per principle and aggregate)
 - |— Identify strengths and weaknesses
 - |— Recommend actions (as Task or Decision issues)
 - |— Store assessment as a graph artifact (not a flat report)

Workflow 4: Gameplay Analysis

When strategic decisions are needed:

1. Review the current landscape map
2. Identify available gameplays
 - |— Which gameplays are enabled by the current landscape?
 - |— Which gameplays are already being played (intentionally or not)?
 - |— Which gameplays are blocked and what would unblock them?
3. For each relevant gameplay
 - |— Describe the play and its expected outcome
 - |— Identify which components are involved
 - |— Assess risks and dependencies
 - |— Map the anticipated landscape shift if the play succeeds
4. Present options
 - |— Create a Decision issue for the Architect with gameplay options
 - |— Include map projections showing each option's impact
 - |— Recommend based on doctrine alignment and strategic fit

Workflow 5: Change Impact Assessment

When a significant change occurs (PR merge, release, new dependency, architectural decision):

1. Identify affected components on active maps

2. Assess positional impact

- └─ Did any component move on the evolution axis?
- └─ Did the dependency graph change?
- └─ Did a new component appear or an old one retire?
- └─ Did the change affect the user-need chain?

3. Produce map diff

- └─ Before/after map snapshots
- └─ Annotated changes with rationale
- └─ Strategic implications (blast radius)

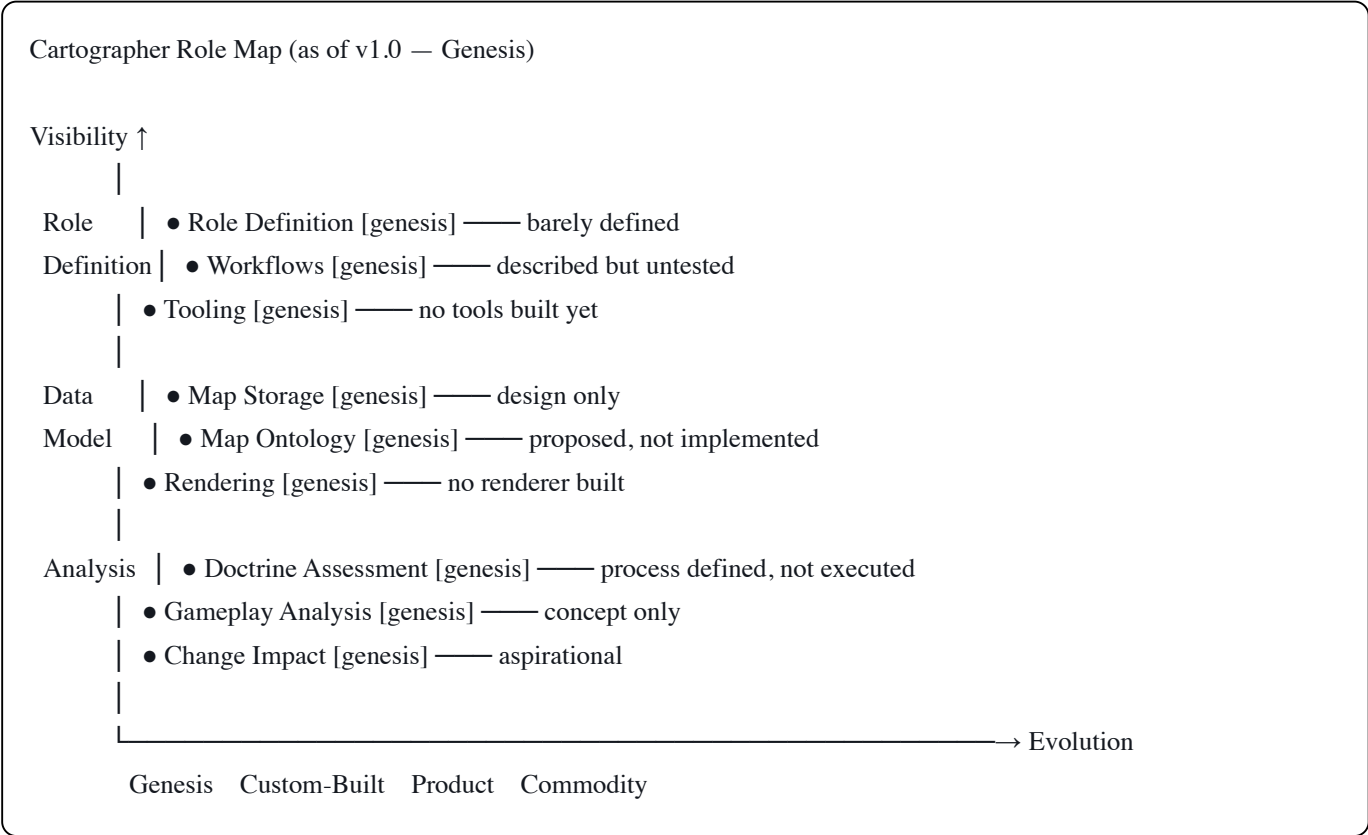
4. Attach to the triggering artifact

- └─ Link the map diff to the PR, release, or Decision issue
- └─ Flag if the change has unexpected strategic implications

Part 6: The Cartographer's Own Map

Mapping the Cartographer

The Cartographer role is itself a component in the landscape. At the time of writing, its map is thin and left-leaning:



Every component sits at Genesis. This is honest. As the role matures, components will move right: the role definition will become Custom-Built (tested and refined), then Product (repeatable and documented). The map

ontology will evolve as we learn what works. The tooling will emerge.

This map is itself the first artifact the Cartographer produces — and its evolution over time is a measure of the role's maturity.

Part 7: File-Based Storage and Issues-FS Integration

Storage Strategy

All map data must be file-based (consistent with the Issues-FS philosophy). The Cartographer's data lives in the graph alongside everything else, but may need additional format support for rendering and interchange:

Primary storage: Issues-FS graph (nodes and edges for components, positions, dependencies, map membership). This is the source of truth. Queryable via MGraph-DB.

Rendering format: Generated visual output (SVG, PNG, or HTML) for human consumption. These are derived artifacts — regenerated from the graph data, not manually maintained.

Interchange format: For compatibility with existing Wardley Mapping tools and communities, the Cartographer may need to export/import in formats like:

- Wardley Map YAML (OnlineWardleyMaps format)
- JSON representations for custom tooling
- OWM (Online Wardley Maps) text format

The conversion between Issues-FS graph representation and interchange formats is the Cartographer's tooling responsibility. The graph is canonical; other formats are projections.

Using Issues-FS to Manage Maps

Maps are managed as Issues-FS issues. This is dogfooding: the Cartographer uses the same tools it maps.

- A new map is a Task issue (type: `Map_Creation`)
- A map update is a Task issue (type: `Map_Update`) linked to the previous version
- A doctrine assessment is a Review_Request (type: `Doctrine_Assessment`)
- A gameplay recommendation is a Decision issue with map evidence
- A change impact assessment is linked to the triggering PR or Release issue

Each of these issue types gets a schema in the Cartographer's role repo, following the same pattern as the other roles.

Part 8: Practical Considerations

Bootstrapping

The Cartographer's first tasks (in priority order):

1. **Define the map ontology** — Node types, link types, evolution stages, axes. Propose to the Architect as a Decision issue. Coordinate with the Librarian for Lexicon integration.
2. **Create the root landscape map** — "What is Issues-FS?" Start with user needs, enumerate components, position them, draw dependencies. This is the Cartographer's first real output.
3. **Map each existing role** — One maturity map per role. These immediately reveal investment gaps and maturity asymmetries.
4. **Run a baseline doctrine assessment** — Score the ecosystem against Wardley doctrine. Surface the most important gaps.
5. **Research tooling** — Evaluate what exists for Wardley Map rendering and storage. Identify what needs to be built vs what can be adopted. Remember: everything must be file-based.
6. **Define the Cartographer's own spec** — Refine this document based on what was learned in steps 1-5.

Tooling Research

The Cartographer should evaluate:

- **Online Wardley Maps (OWM)** — Open-source browser-based tool. Text-based map format. Could serve as a rendering reference.
- **MapScript / Wardley Map Canvas** — Alternative tooling for map creation.
- **Custom rendering** — Given Issues-FS's graph-native architecture, a custom renderer that reads map data from the graph and produces SVG may be more aligned than adopting an external tool.
- **MGraph-DB integration** — Map queries as graph queries. "Show me all components at Genesis" is a graph traversal, not a file search.

Measuring the Cartographer's Value

The Cartographer's value is measured not by map count but by decision quality. Indicators:

- **Decision issues that reference maps** — Are strategic decisions being made with landscape context?
- **Doctrine score trend** — Is the ecosystem's doctrine health improving over time?
- **Map currency** — Are maps being maintained (regular updates) or stale?
- **Surprise reduction** — Are there fewer "we didn't see that coming" moments?
- **Gameplay execution** — Are identified gameplays being pursued and tracked?

Decisions Log

#	Decision	Rationale
CAR1	Name is Cartographer, not Mapper or Strategist	Cartographer implies the creation and maintenance of maps as primary artifacts. Mapper is too generic. Strategist implies decision-making authority that belongs to the Architect and Conductor. The Cartographer provides the landscape; others decide what to do about it.

#	Decision	Rationale
CAR2	Maps are graphs with positional metadata, not separate artifacts	Storing maps as graph data (nodes + evolution edges + map membership edges) keeps maps integrated with the ecosystem. Map queries are graph queries. Map diffs are graph diffs. This avoids a parallel data silo.
CAR3	Multiple evolution axes, not just the standard one	The standard Genesis→Commodity axis is essential but not sufficient. Alternative axes (openness, automation, documentation, test coverage, graph connectivity) reveal different strategic dimensions of the same landscape.
CAR4	Maps of maps (fractal)	Every component on a map can drill down into its own map. This mirrors the fractal scope principle from thinking-in-graphs and enables analysis at any level of zoom.
CAR5	Doctrine and gameplays are first-class responsibilities	Most Wardley Mapping practitioners stop at the map. The Cartographer must go further: doctrine assessment gives strategic health checks; gameplay analysis identifies available moves. These are where maps become actionable.
CAR6	File-based storage, Issues-FS as primary tool	Consistent with the ecosystem philosophy. Map data stored in graph files, rendered on demand. The Cartographer dogfoods Issues-FS to manage its own work.
CAR7	Change impact mapping as an eventual capability	Producing map diffs for every PR is aspirational. It requires mature tooling and established maps. The Cartographer should build toward this incrementally, starting with manual diffs for significant changes.

Side-Capture: Ideas for Further Exploration

The following ideas emerged during the definition of this role but need separate treatment:

1. Source Code Mapping

The transcript envisions a future where every PR comes with an updated map and a diff showing blast radius. This requires:

- Mapping individual code modules as components with evolution positions
- Automated detection of which map components a code change affects
- Integration with the DevOps pipeline to generate map diffs at PR time

This is a significant capability that deserves its own design document once the Cartographer's basic workflows are established.

2. Daily Situational Awareness via Maps

The idea that "my daily status should be provided by a map" — showing interdependencies, blockers, and progress spatially rather than as a list. This could be a dashboard view that the Conductor uses for daily standup, rendered from the current state of active maps.

3. Competitive Intelligence Maps

Systematic comparison of Issues-FS against GitHub Issues, Jira, Linear, etc. using shared user needs and different component positions. This could be a recurring Cartographer activity that feeds into product strategy.

References

- [Thinking in Graphs: Meaning Through Connectivity](#) — Foundational philosophy
 - [Issues-FS Lexicon Architecture v2.0](#) — The root graph
 - [Issues-FS Role-Based Agent Coordination](#) — The six-role model
 - [Issues-FS Librarian Role](#) — Complementary knowledge curation role
 - [Issues-FS Role Architecture Framework Analysis](#) — Framework stress-test (includes initial Wardley analysis)
 - [Wardley Maps](#) — Simon Wardley's mapping methodology
 - [Online Wardley Maps](#) — Open-source mapping tool
-

Issues-FS Cartographer Role v1.0

Date: 2026-02-07